VSP TECHVERSE
An Institute of AI & Robotics

Machine Learning for AI & Robotics



Supervised, Unsupervised & Reinforcement Learning · Linear, Polynomial, Lasso & Ridge Regression

VSP TECHVERSE
An Institute of AI & Robotics

What is Machine Learning?

Machine Learning (ML) is a branch of Artificial Intelligence where computers learn patterns from data and improve at a task with experience, without being explicitly programmed with every rule.

- The model learns from examples (training data)
- It finds the relationship between inputs and outputs
- It then makes predictions or decisions on new data
- More good data usually means better performance

TRADITIONAL PROGRAMMING



MACHINE LEARNING



In ML, the computer works out the rules itself from examples.

ROBOTICS EXAMPLE Instead of hand-coding a rule for every obstacle, a robot learns to recognise obstacles from thousands of labelled camera images.

Types of Machine Learning



Supervised Learning

Learns from labelled data: every input comes with the correct answer.

GOAL
Predict an output for new inputs

ROBOTICS EXAMPLE
Predicting battery drain from payload



Unsupervised Learning

Learns from unlabelled data: no answers are given.

GOAL
Find hidden patterns or groups

ROBOTICS EXAMPLE
Grouping terrain types from wheel sensor data



Reinforcement Learning

Learns by trial and error, receiving rewards for good actions.

GOAL
Learn the best actions to take

ROBOTICS EXAMPLE
A robot arm learning to grasp objects

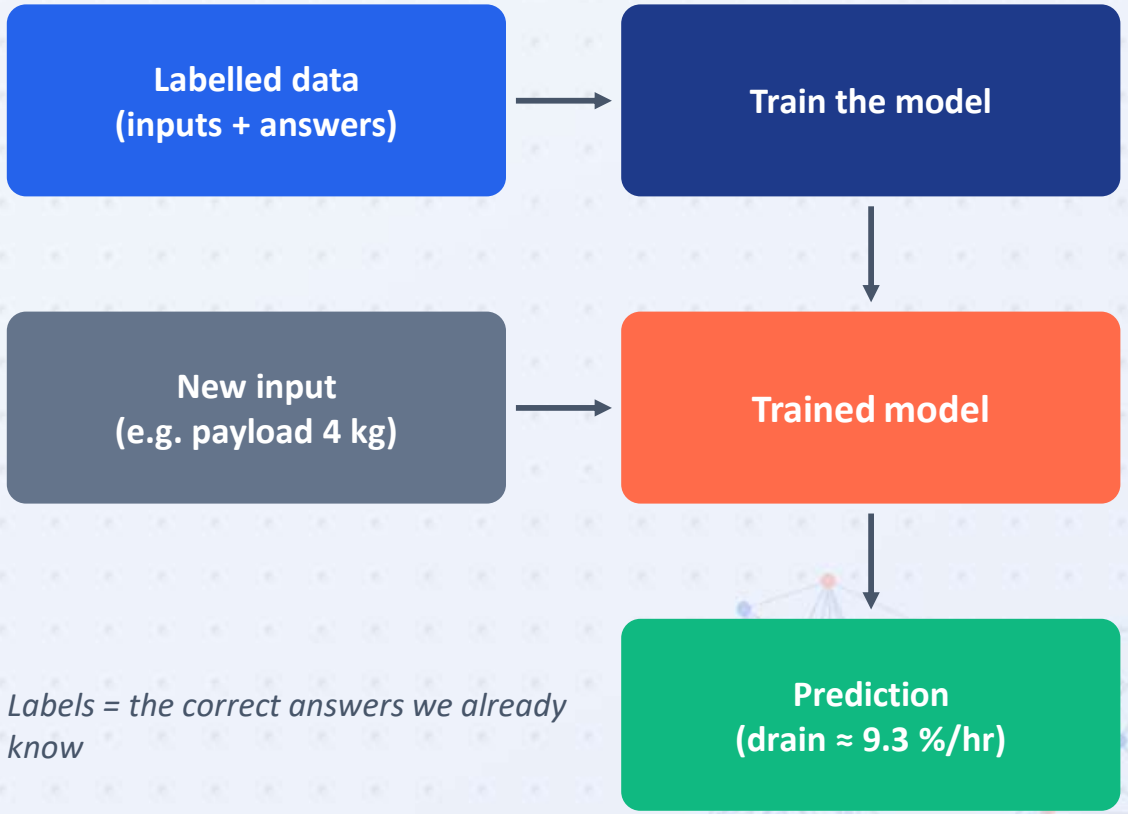
Supervised Learning

The model learns from labelled examples, where each input (x) is paired with the correct output (y). After training, it predicts y for new inputs.

- **Classification:** predicts a category, e.g. obstacle / no obstacle
- **Regression:** predicts a number, e.g. battery drain in % per hour
- **Common models:** linear regression, decision trees, neural networks
- **Needs:** enough correctly labelled data

ROBOTICS EXAMPLE A delivery robot learns to predict its battery drain from payload, using past trips where the drain was measured.

HOW IT WORKS

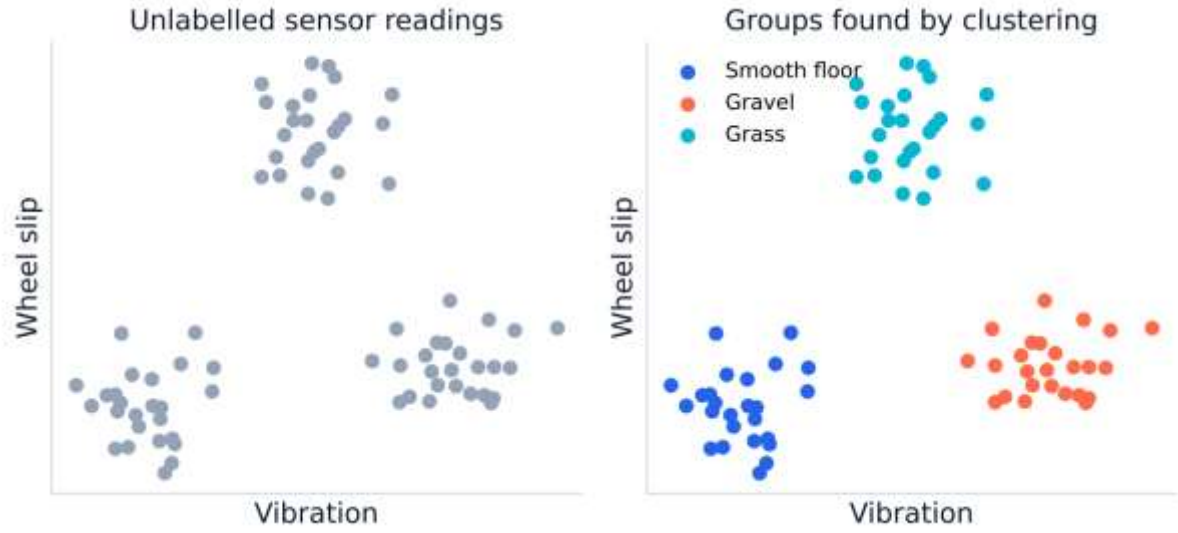


Labels = the correct answers we already know

Unsupervised Learning

The model is given data without labels and finds hidden structure on its own, such as groups of similar readings.

- **Clustering:** groups similar data points (e.g. K-Means)
- **Dimensionality reduction:** compresses many sensor signals into a few
- **Anomaly detection:** spots readings that do not fit any pattern
- **No labels needed,** so it works on raw sensor logs



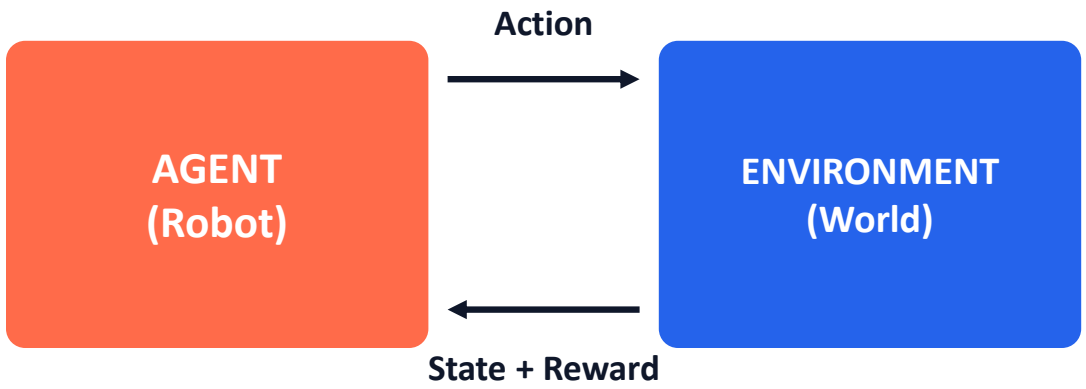
The algorithm discovers the three groups; we only name them afterwards.

ROBOTICS EXAMPLE A rover groups its wheel-slip and vibration readings into terrain types, or flags unusual motor vibration before a failure.

Reinforcement Learning

An agent learns by interacting with its environment: it takes actions, receives rewards or penalties, and gradually learns the actions that earn the most reward.

- **Agent:** the learner, e.g. the robot
- **Environment:** the world the robot acts in
- **State:** what the robot observes right now
- **Action:** what the robot decides to do
- **Reward:** feedback: + for good, – for bad
- **Policy:** the strategy it learns for choosing actions



The loop repeats thousands of times while the robot improves.

ROBOTICS EXAMPLE A robot arm learns to grasp: +1 reward for a firm grip, −1 for dropping the object.

Our Dataset: robot_battery_data.csv

50 trips by a delivery robot. Battery drain rises in a straight line with payload and speed, so it is ideal for linear regression. Every example in this deck uses this file.

payload_kg	speed_mps	motor_temp_c	light_lux	motor_current_a	battery_drain
3.98	1.0	38.6	390	4.36	8.83
2.47	0.52	35.5	761	2.97	5.74
4.36	2.47	29.0	107	4.67	10.55
3.64	2.16	39.5	816	4.08	9.27
0.92	2.26	36.7	534	1.69	6.07
4.89	2.45	35.8	212	5.21	11.95

First 6 of 50 rows

payload_kg
Payload carried (kg)

speed_mps
Driving speed (m/s)

motor_temp_c
Motor temperature (°C), no real effect

light_lux
Ambient light (lux), no real effect

motor_current_a
Motor current (A), rises with payload

battery_drain
TARGET: battery drain (% per hour)

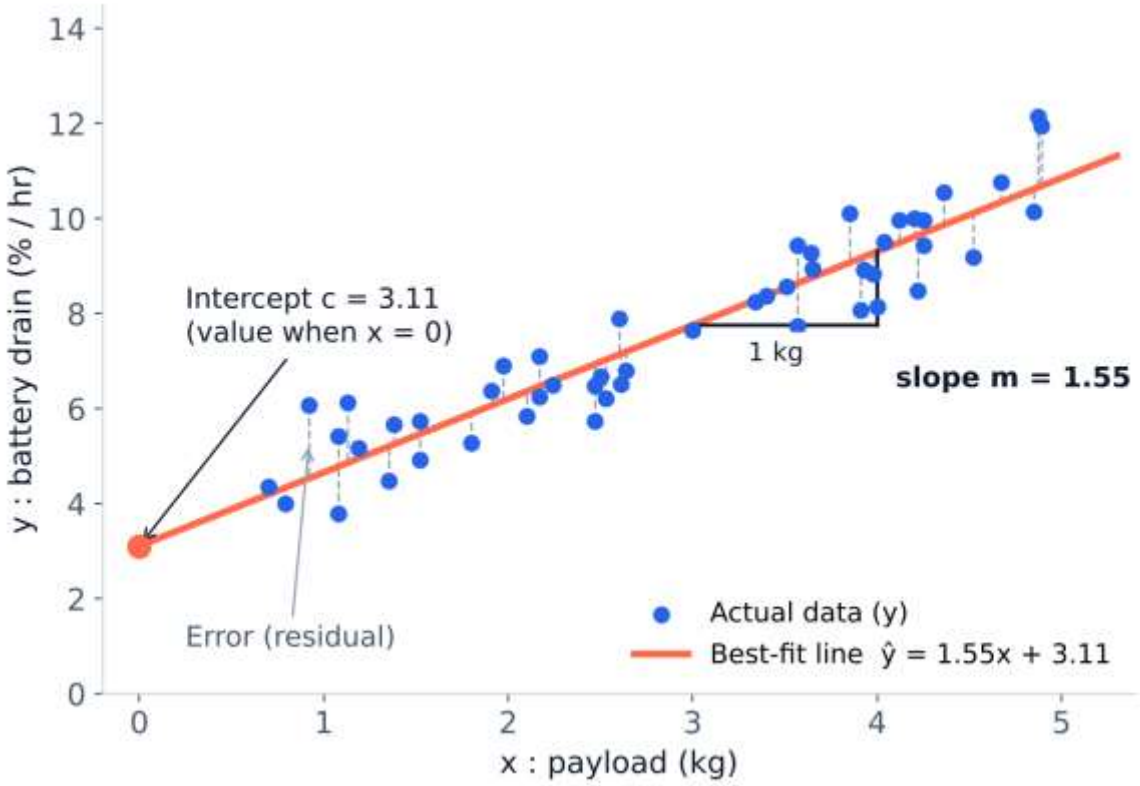
Simple Linear Regression

Predicts a numeric output (y) from one input (x) by fitting the best straight line through the data.

EQUATION

$$\hat{y} = m\,x + c$$

- **x:** input feature (payload in kg)
- **ŷ:** predicted output (battery drain)
- **m:** slope, change in y for 1 unit of x
- **c:** intercept, value of y when x = 0
- **Best fit:** the line with the smallest total squared error (least squares)



Residual = actual y − predicted ŷ. The model makes these errors as small as possible.

Simple Linear Regression in Python

model.py

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

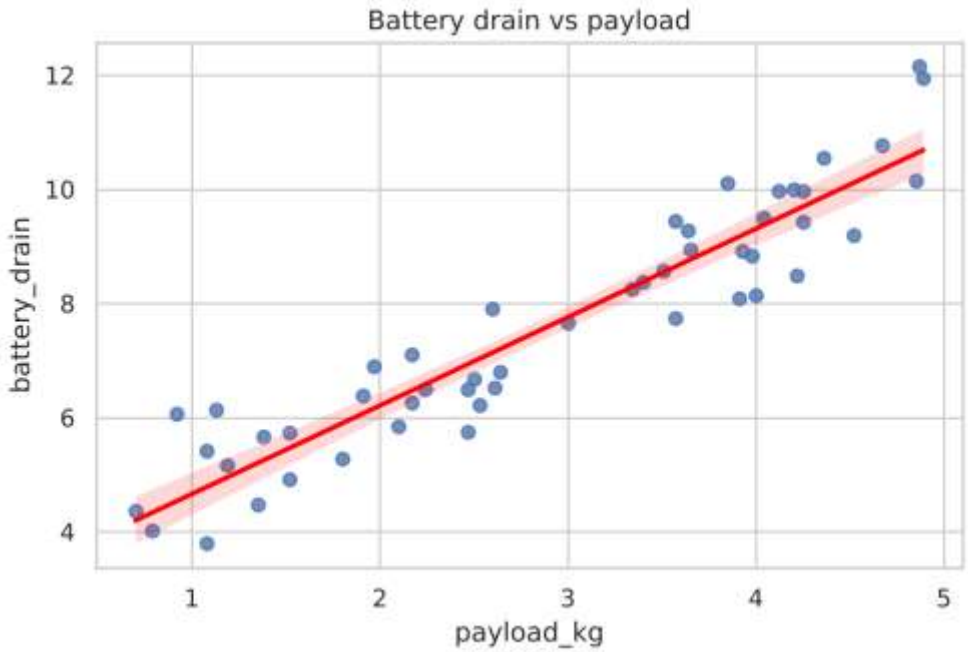
df = pd.read_csv("robot_battery_data.csv")
X = df[["payload_kg"]]
y = df["battery_drain"]

model = LinearRegression().fit(X, y)
print("Slope:", round(model.coef_[0], 2))
print("Intercept:", round(model.intercept_, 2))
new = pd.DataFrame({"payload_kg": [4.0]})
print("Drain at 4 kg:", round(model.predict(new)[0], 2))

sns.set_theme(style="whitegrid")
sns.regplot(data=df, x="payload_kg", y="battery_drain",
            line_kws={"color": "red"})
plt.title("Battery drain vs payload")
plt.show()
```

OUTPUT

```
Slope: 1.55
Intercept: 3.11
Drain at 4 kg: 9.31
```



- **fit() / predict()** learn the line and use it for new payloads
- **sns.regplot()** draws the data and the fitted line; the band is the 95% confidence interval
- **Result:** every extra kg adds about 1.55 % per hour of drain

Evaluation Metrics for Regression

How good is our model? Compare its predictions (ŷ) with the true values (y) on test data the model has never seen.

MAE

Mean Absolute Error

$$MAE = (1/n)\ \Sigma|y - \hat{y}|$$

Average size of the errors, in the same unit as y.

Lower is better

ROBOT EXAMPLE

Off by 0.5 % per hour on average

MSE

Mean Squared Error

$$MSE = (1/n)\ \Sigma(y - \hat{y})^2$$

Squares each error, so big mistakes count much more.

Lower is better

ROBOT EXAMPLE

Punishes one badly wrong battery forecast

RMSE

Root Mean Squared Error

$$RMSE = \sqrt{MSE}$$

Square root of MSE, back in the unit of y.

Lower is better

ROBOT EXAMPLE

Typical error of about 0.6 % per hour

R^2

R-squared score

$$R^2 = 1 - SSE / SST$$

Share of the variation explained: SSE = error left, SST = total variation. 1 is perfect.

Higher is better (max 1)

ROBOT EXAMPLE

R^2 = 0.9 explains 90% of the variation

Train / test split: train on about 80% of the data and measure the metrics on the other 20%, so we check how the model does on new data.

Evaluation Metrics in Python

model.py

```python
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (mean_absolute_error,
                             mean_squared_error, r2_score)

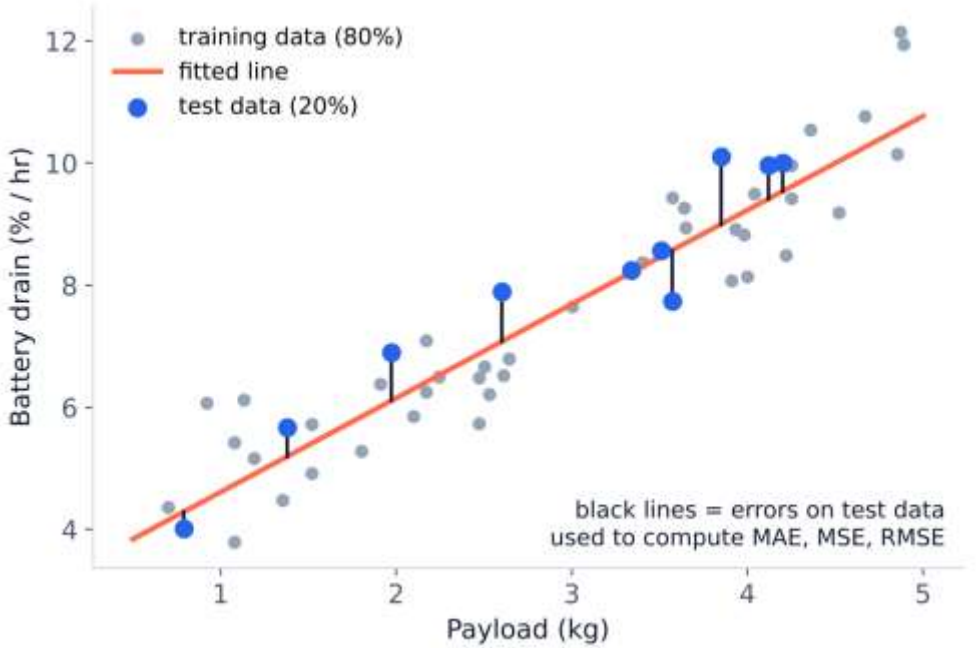
df = pd.read_csv("robot_battery_data.csv")
X = df[["payload_kg"]]
y = df["battery_drain"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression().fit(X_train, y_train)
y_pred = model.predict(X_test)

mse = mean_squared_error(y_test, y_pred)
print("MAE :", round(mean_absolute_error(y_test, y_pred), 3))
print("MSE :", round(mse, 3))
print("RMSE:", round(np.sqrt(mse), 3))
print("R2  :", round(r2_score(y_test, y_pred), 3))
```

OUTPUT

```
MAE : 0.545
MSE : 0.406
RMSE: 0.637
R2  : 0.885
```



- **train_test_split** keeps 20% of the trips aside for testing
- **MAE 0.55, RMSE 0.64:** predictions are off by about 0.6 % per hour
- **R^2 0.885:** the model explains about 89% of the variation on new data

Multiple Linear Regression

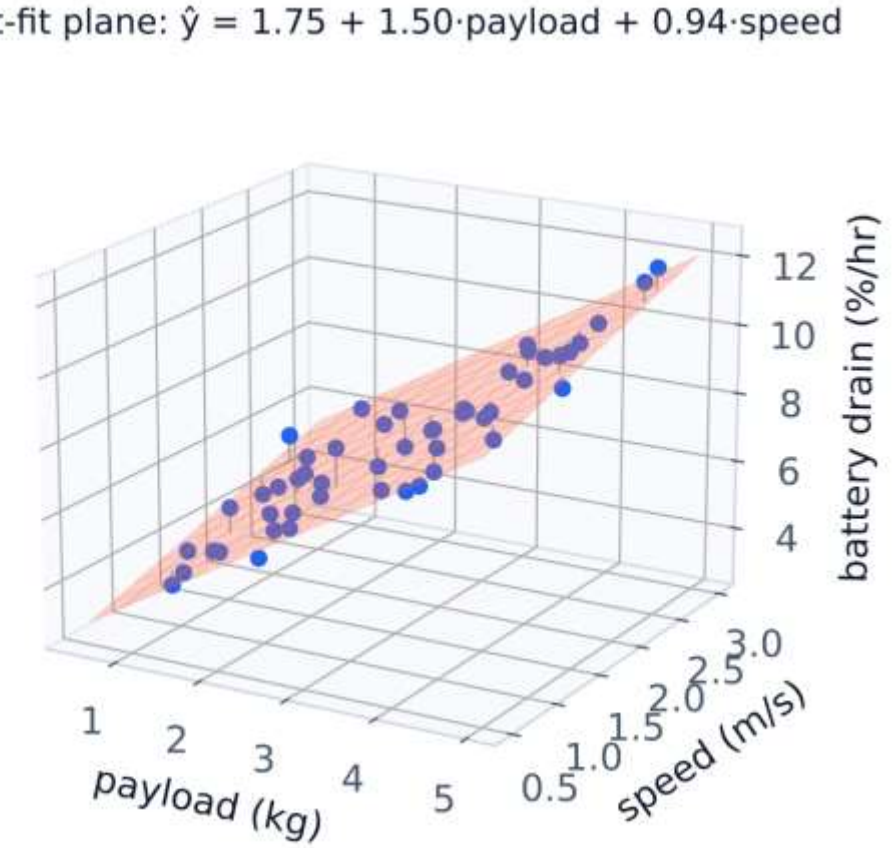
Uses two or more inputs to predict one output. Each input gets its own coefficient (weight), so the best-fit line becomes a best-fit plane.

EQUATION

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \ldots + b_nx_n$$

- **b_0:** intercept
- **b_1 … b□:** effect of each input when the others stay fixed
- **Here:** x_1 = payload, x_2 = speed
- **Fit:** least squares, the same idea as simple linear regression

ROBOTICS EXAMPLE Battery drain depends on payload and speed together.



Each dot is one trip; grey lines are the errors to the plane.

Multiple Linear Regression in Python

model.py

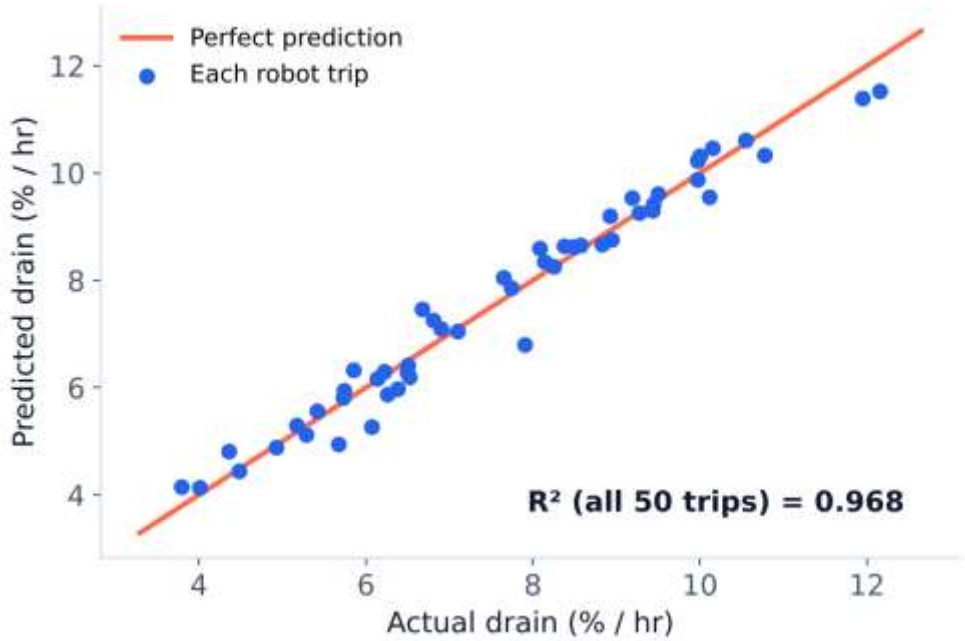
```python
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics

df = pd.read_csv("robot_battery_data.csv")
X = df[["payload_kg", "speed_mps", "motor_temp_c"]]
y = df["battery_drain"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = LinearRegression().fit(X_train, y_train)
print("Coefficients:", model.coef_.round(2))
print("Intercept:", round(model.intercept_, 2))
y_pred = model.predict(X_test)
mse = metrics.mean_squared_error(y_test, y_pred)
print("MAE :", round(metrics.mean_absolute_error(y_test, y_pred), 3))
print("RMSE:", round(np.sqrt(mse), 3))
print("R2  :", round(metrics.r2_score(y_test, y_pred), 3))
```

OUTPUT

```
Coefficients: [1.51 0.98 0.01]
Intercept: 1.27
MAE : 0.336
RMSE: 0.505
R2  : 0.928
```



- **train_test_split** trains on 80% of the trips, tests on 20%
- **Coefficients:** payload 1.51, speed 0.98, motor temp ≈ 0
- **On test data:** R^2 0.928, RMSE 0.51, better than payload alone (R^2 0.885)

Advantages & Disadvantages

ADVANTAGES

- Simple to understand, explain and visualise
- Fast to train, even on a robot's small onboard computer
- Coefficients show how much each input affects the output
- Works well when the relationship is roughly linear
- A good baseline to compare other models against

DISADVANTAGES

- Assumes a straight-line relationship between inputs and output
- Sensitive to outliers, like a single faulty sensor reading
- Unstable when inputs are strongly correlated (multicollinearity)
- Can overfit when there are many features
- Cannot capture complex, curved patterns on its own

Polynomial Regression

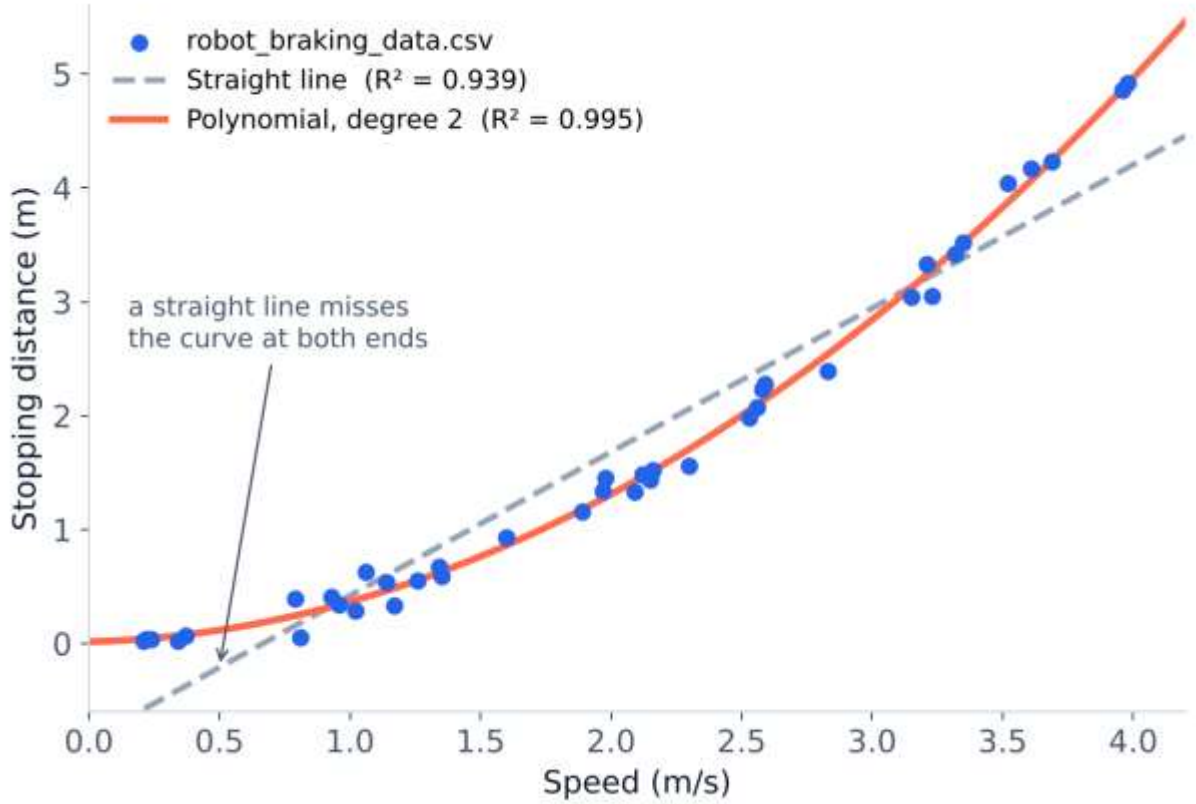
When the data curves, add powers of the input (x^2, x^3 …) as new features. The model is still linear in its coefficients, so the same LinearRegression fits a curve.

DEGREE 2 EQUATION

$$\hat{y} = b_0 + b_1 x + b_2 x^2$$

- **Degree:** the highest power; degree 2 makes one bend
- **Our fit:** $\hat{y} = 0.015 + 0.057x + 0.295x^2$
- **Careful:** a very high degree overfits the training data

ROBOTICS EXAMPLE A robot's stopping distance grows with the square of its speed (robot_braking_data.csv).



The curve explains 99.5% of the variation; the straight line only 93.9%.

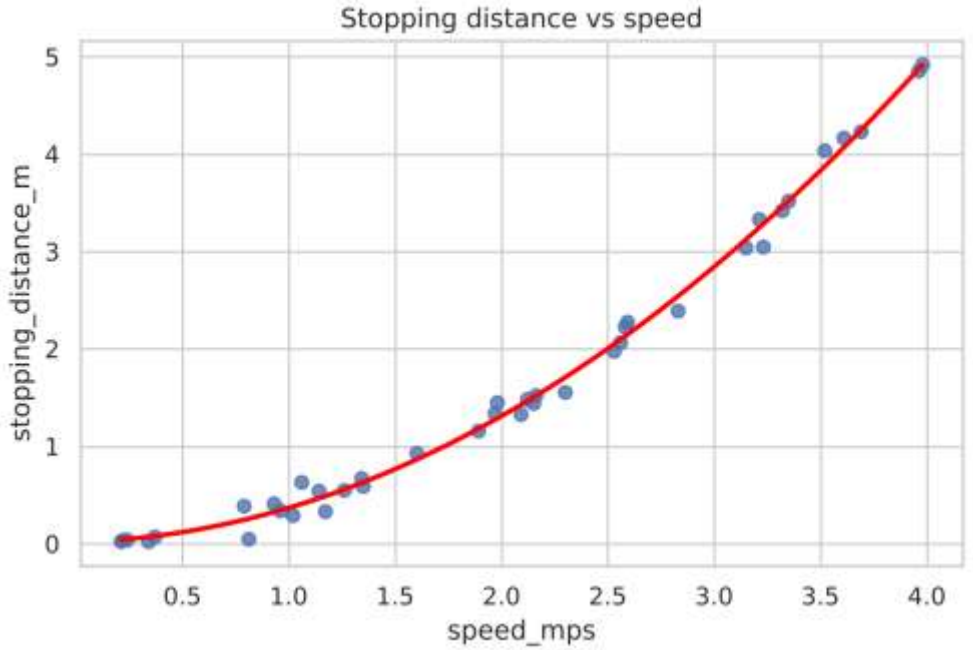
Polynomial Regression in Python

model.py

```python
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split
from sklearn import metrics

df = pd.read_csv("robot_braking_data.csv")
X, y = df[["speed_mps"]], df["stopping_distance_m"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
poly = make_pipeline(PolynomialFeatures(degree=2),
                     LinearRegression()).fit(X_train, y_train)
y_pred = poly.predict(X_test)
mse = metrics.mean_squared_error(y_test, y_pred)
print("MAE :", round(metrics.mean_absolute_error(y_test, y_pred), 3))
print("RMSE:", round(np.sqrt(mse), 3))
print("R2  :", round(metrics.r2_score(y_test, y_pred), 3))

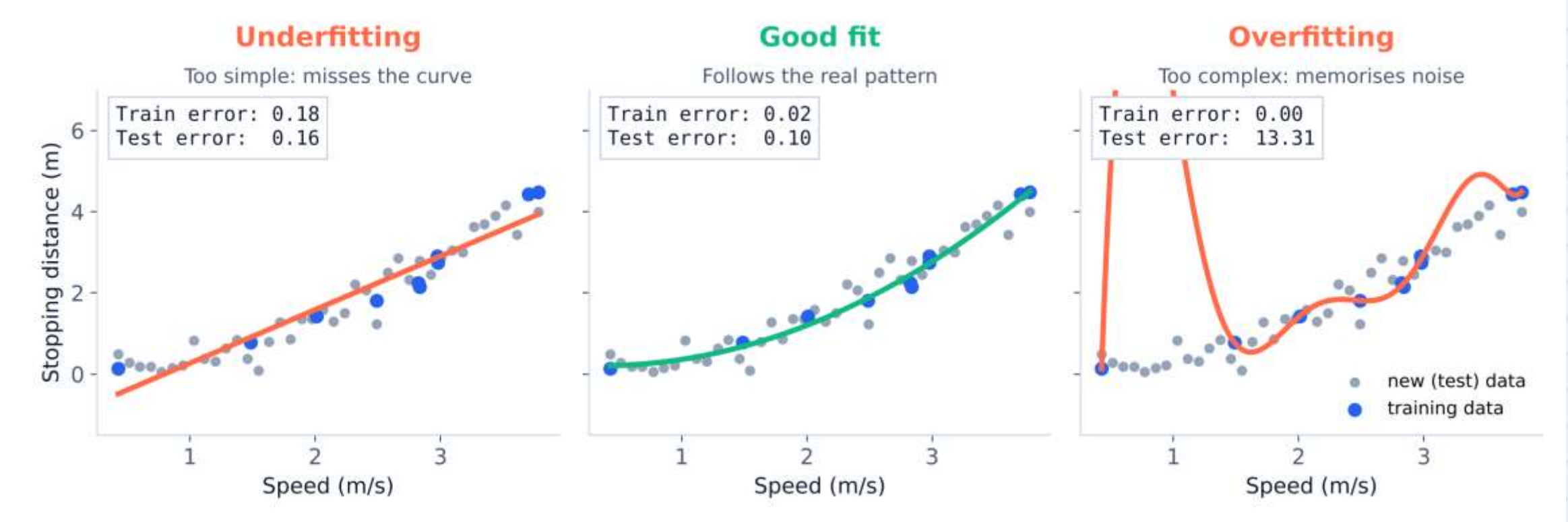
sns.set_theme(style="whitegrid")
sns.regplot(data=df, x="speed_mps", y="stopping_distance_m",
            order=2, ci=None, line_kws={"color": "red"})
plt.title("Stopping distance vs speed")
plt.show()
```



OUTPUT

```
MAE : 0.075
RMSE: 0.097
R2  : 0.994
```

Underfitting vs Overfitting



UNDERFITTING
Model is too simple to capture the pattern. High error on training AND new data. Fix: use a more complex model, e.g. polynomial.

GOOD FIT
Model learns the real pattern, not the noise. Low error on both training and new data. This is the goal.

OVERFITTING
Model is too complex and memorises noise. Almost zero training error but large error on new data. Fix: more data, a simpler model, or regularization.

Regularization: The Fix for Overfitting

Regularization adds a penalty for large coefficients to the loss. The model can no longer bend wildly to fit noise, so it generalises better to new data.

REGULARIZED LOSS

$$\text{Loss} = \text{Error} + \alpha \times \text{Penalty}$$

LASSO (L1)

$\text{Penalty} = \alpha\, \Sigma|b_i|$

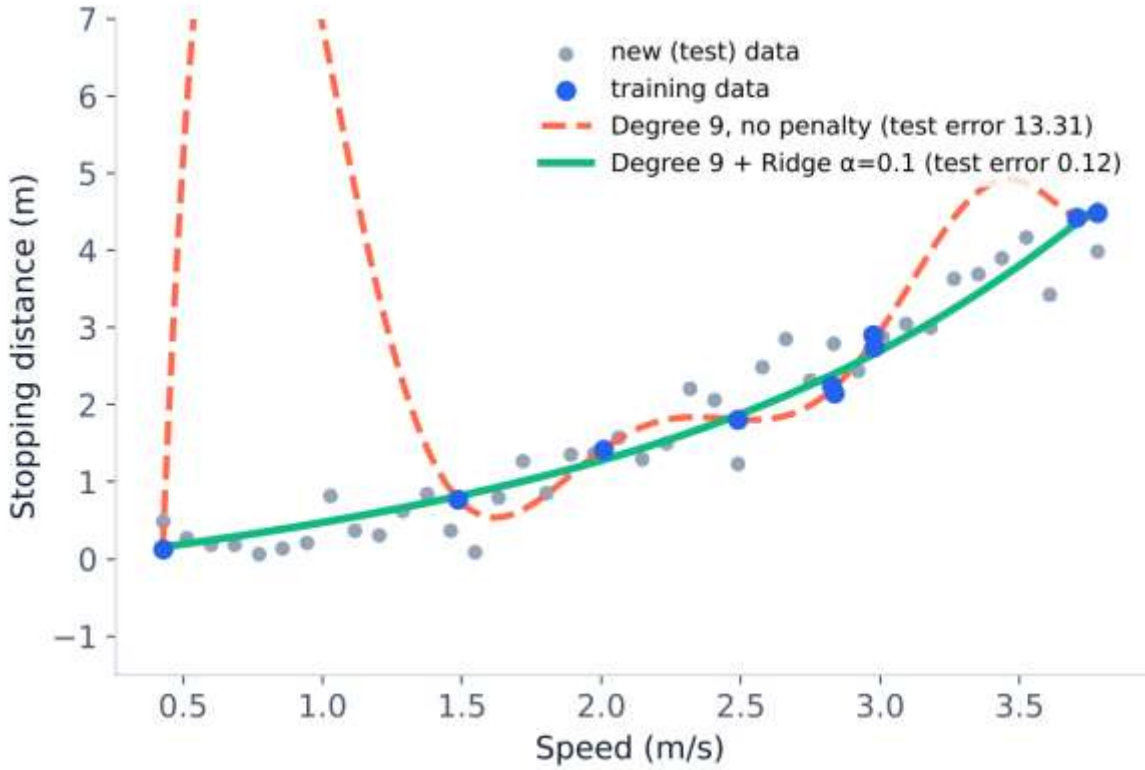
Shrinks some coefficients to exactly 0

RIDGE (L2)

$\text{Penalty} = \alpha\, \Sigma b_i^2$

Shrinks all coefficients, never to 0

- **α = 0:** plain linear regression, no penalty
- **Larger α:** simpler, smoother model
- **Too large α:** the model underfits, so choose α carefully



Same degree-9 model: Ridge cuts the error on new data from 13.31 to 0.12.

Lasso Regression (L1)

A regularization technique: linear regression plus a penalty on the absolute size of the coefficients. It reduces overfitting and can push unhelpful coefficients to exactly zero.

COST FUNCTION

$$\text{Loss} = \Sigma(y - \hat{y})^2 + \alpha\, \Sigma|b_i|$$

WHAT IT DOES

- **Removes features:** weak coefficients become exactly 0
- **Simpler model:** keeps only the inputs that really matter
- **α (alpha):** bigger α removes more features; α = 0 is plain linear regression

WHEN TO USE IT

- You have many sensor inputs but expect only a few to matter
- You want automatic feature selection and an easy-to-explain model
- Avoid it when inputs are strongly correlated: it keeps one and drops the rest

model.py

```python
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Lasso
from sklearn import metrics

df = pd.read_csv("robot_battery_data.csv")
cols = ["payload_kg", "speed_mps", "motor_temp_c", "light_lux"]
X, y = df[cols], df["battery_drain"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

lasso = Lasso(alpha=0.2).fit(X_train, y_train)
print("Coefficients:", lasso.coef_.round(3))
y_pred = lasso.predict(X_test)
mse = metrics.mean_squared_error(y_test, y_pred)
print("MAE :", round(metrics.mean_absolute_error(y_test, y_pred), 3))
print("RMSE:", round(np.sqrt(mse), 3))
print("R2  :", round(metrics.r2_score(y_test, y_pred), 3))
```

OUTPUT

```
Coefficients: [1.403 0.531 0.    0.   ]
MAE : 0.404
RMSE: 0.505
R2  : 0.928
```

Ridge Regression (L2)

A regularization technique: linear regression plus a penalty on the squared size of the coefficients. It reduces overfitting by shrinking every coefficient, never to zero.

COST FUNCTION

$$\text{Loss} = \Sigma(y - \hat{y})^2 + \alpha\, \Sigma b_i^2$$

WHAT IT DOES

- **Keeps all features:** coefficients shrink but never reach 0
- **Stable weights:** shares weight between correlated inputs
- **α (alpha):** bigger α means smaller coefficients; α = 0 is plain linear regression

WHEN TO USE IT

- Inputs are strongly correlated, like payload and motor current
- Most features are useful and you want to keep them all
- Plain linear regression gives large, unstable weights or overfits

model.py

```python
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn import metrics

df = pd.read_csv("robot_battery_data.csv")
# motor current rises almost exactly with payload
X = df[["payload_kg", "motor_current_a"]]
y = df["battery_drain"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

ridge = Ridge(alpha=1.0).fit(X_train, y_train)
print("Coefficients:", ridge.coef_.round(2))
y_pred = ridge.predict(X_test)
mse = metrics.mean_squared_error(y_test, y_pred)
print("MAE :", round(metrics.mean_absolute_error(y_test, y_pred), 3))
print("RMSE:", round(np.sqrt(mse), 3))
print("R2  :", round(metrics.r2_score(y_test, y_pred), 3))
```

OUTPUT

```
Coefficients: [0.85 0.75]
MAE : 0.556
RMSE: 0.642
R2  : 0.883
```

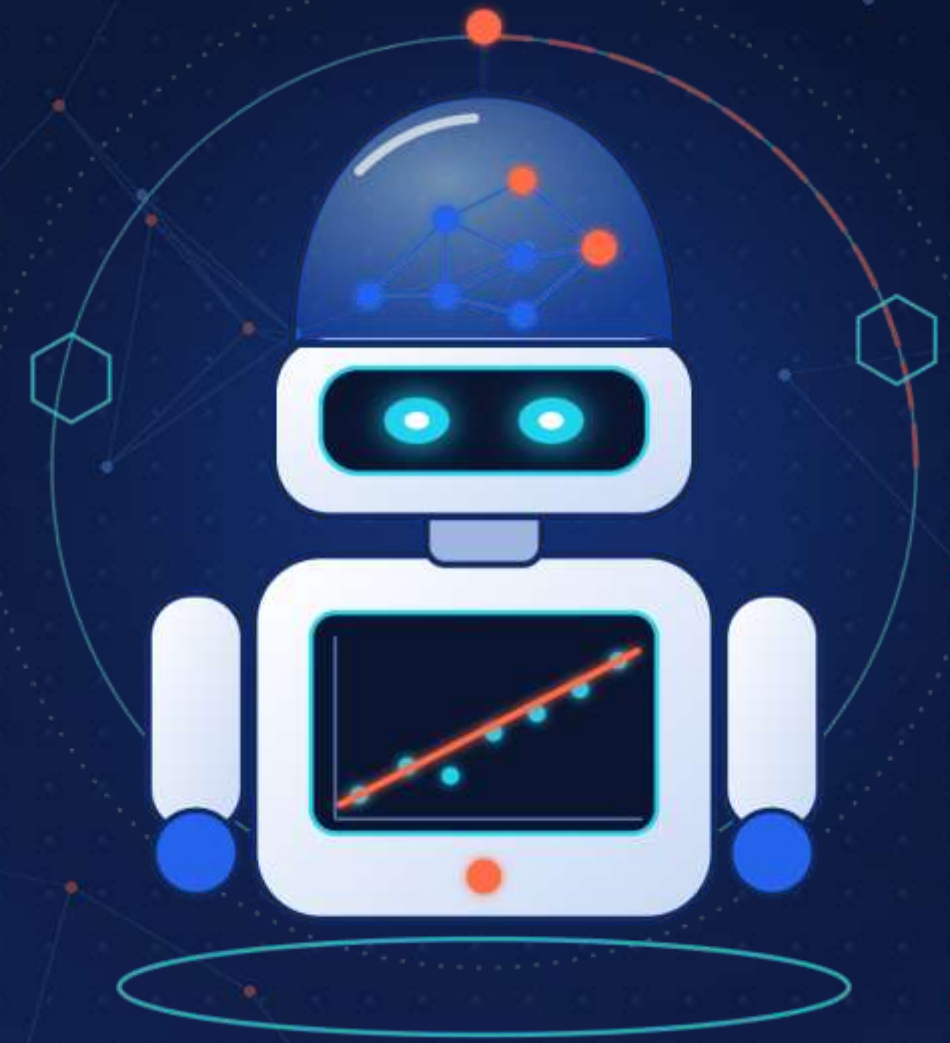
Linear vs Lasso vs Ridge

	Linear Regression	Lasso (L1)	Ridge (L2)
Penalty	None	$\alpha \Sigma \lvert b_i \rvert$	$\alpha \Sigma b_i^2$
Coefficients	Unrestricted	Some become exactly 0	Shrunk, never 0
Reduces overfitting	No	Yes	Yes
Feature selection	No	Yes	No
Correlated inputs	Unstable	Keeps one, drops others	Handles well
Best for	Few, independent inputs	Many inputs, few useful	Many useful, correlated inputs

VSP TECHVERSE
An Institute of AI & Robotics

Thank You

Keep learning, keep building



VSP TECHVERSE
An Institute of AI & Robotics